Web Denetim Merkezi

Teknik Doküman

Programın amacı, mimarisi, süreçleri, sonuçların nasıl elde edildiği ve kod haritası.

Sürüm 1.1 · Ekim 2026
github.com/mehmetfatihvar/web-denetim-merkezi
Mersin Üniversitesi staj çalışması
Kullanım için bkz. Kullanım Kılavuzu (docs/KULLANIM_KILAVUZU.pdf)

İçindekiler

1. Giriş **4**
1.1 Bu doküman nedir? 4
1.2 Programın amacı ve hizmet ettiği ihtiyaç 4
1.3 Arka plan 4
1.4 Temel kavramlar 5
2. Genel bakış **6**
2.1 Program hangi soruları cevaplar? 6
2.2 Tasarım ilkeleri 6
2.3 Kullanılan teknolojiler 7
3. Mimari **8**
3.1 Katmanlar 8
3.2 Klasör yapısı 9
3.3 Bir adımın yaşam döngüsü 9
4. Uçtan uca süreç **10**
4.1 Süreç haritası 10
4.2 Hazırlık 10
4.3 Site keşfi 11
4.4 Kanıtlı test 11
4.5 Belge ve kırık link denetimi 12
4.6 Şablon kümeleme ve görsel/UI kusur denetimi 13
4.7 Kırık sayfa doğrulama (teyit) 13
4.8 Yapay zekâ destekli görsel denetim 13
4.9 Doğrula ve düzelt 16
4.10 Bologna aktarımı 16
4.11 Teslim paketi 17
5. Sonuçlar nasıl elde ediliyor? **18**
5.1 Sonuç → süreç → dosya 18
5.2 Güven katmanları 18
5.3 Sayılara dair örnek: ÖİDB 19
6. Kod haritası: çekirdek (merkez/) **20**
6.1 Bir adımın tanımı 21
7. Kod haritası: araçlar (tools/) **22**
7.1 Keşif, test ve doğrulama araçları 22
7.2 Görsel denetim hattı (tools/visual_audit/) 22
8. Kod haritası: Bologna (bologna-scraper/) **24**
9. Çalıştırma motoru ve arayüz **25**
9.1 Akış 25
9.2 Alt süreç ve kayıt 25
9.3 İlerleme tanıma 25
9.4 Kalan süre tahmini 25
9.5 Hata ve uyarı tanıma 26
9.6 Durdurma ve devam 26
9.7 Arayüz performansı 26

9.8 Adım raporları ... 26

10. Veri ve dosya biçimleri ... 27

10.1 Çıktı klasörü ... 27

10.2 Ana dosyalar ... 27

11. Ayarlar ve profiller ... 29

12. Kalite güvencesi ... 30

12.1 Testler ... 30

12.2 Sürekli entegrasyon ... 30

12.3 Dokümantasyon ... 30

13. Sınırlar ve geliştirme önerileri ... 31

13.1 Bilinen sınırlar ... 31

13.2 Geliştirme önerileri ... 31

Ek A. Adım kataloğu ... 32

Ek B. Sözlük ... 33

1. Giriş

1.1 Bu doküman nedir?

Bu doküman **Web Denetim Merkezi** programının ne işe yaradığını, hangi parçalardan oluştuğunu, her sonucun hangi süreçlerden geçerek elde edildiğini ve hangi kod parçasının neyi temsil ettiğini anlatır. Programın **nasıl kullanılacağı** (düğmeler, ayarlar, adım adım iş akışı) ayrı bir belgede, **Kullanım Kılavuzu**'nda (`docs/KULLANIM_KILAVUZU.pdf`) anlatılır. Bu doküman ise programın **içini** anlatır: neden böyle tasarlandığını, verinin nereden gelip nereye gittiğini ve sonuçlara ne kadar güvenilebileceğini.

Doküman üç tür okuyucu için yazıldı:

- **Sonuçları değerlendirecek kişi** (ör. kurumun web sorumlusu, staj değerlendiricisi): 2. bölüm (genel bakış), 4. bölüm (süreçler) ve 5. bölüm (hangi sonuç nereden geliyor, ne kadar güvenilir).
- **Programı başka bir siteye uygulayacak kişi**: 2, 4, 10 ve 11. bölümler.
- **Programı geliştirecek ya da bakımını yapacak kişi**: 3. bölüm (mimari), 6–9. bölümler (kod haritası) ve 12–13. bölümler (kalite güvencesi, sınırlar).

1.2 Programın amacı ve hizmet ettiği ihtiyaç

Büyük bir kurum sitesi (ör. bir üniversitenin binlerce sayfalık bilgi sitesi) zamanla bozulur: sayfalar taşınır ve eski bağlantılar kırılır, belgeler silinir, içerik girilirken tasarım bozulur, mobilde yazılar ekrandan taşar, Türkçe karakterler bozulur. Bu sorunları sayfa sayfa elle bulmak binlerce sayfada pratik olarak imkânsızdır; bulunsa bile "kaç sayfada var, hangisi önce düzeltilmeli, gerçekten var mı?" soruları cevapsız kalır.

Web Denetim Merkezi bu ihtiyaca şu şekilde hizmet eder:

- **Eksiksiz kapsam:** Sitenin tıklanabilir bütün sayfalarını kendisi keşfeder ve hepsini gerçek bir tarayıcıda açar. Mersin Üniversitesi ÖİDB sitesinde bu 17.394 sayfa, 36.899 tekil bağlantı ve yaklaşık 34.000 ekran görüntüsü demektir.
- **Kanıt:** Her sayfa için masaüstü ve mobil tam sayfa ekran görüntüsü alır. Her bulgu bir dosyaya, bir satıra ve çoğu zaman bir görüntüye bağlıdır; "bana öyle geldi" değil "işte burada" denebilir.
- **Doğrulama:** Otomatik araçlar yanlış alarm üretir. Program bulguları ikinci, bağımsız bir yolla yeniden sınar (doğrudan erişim, gerçek tıklama, canlı DOM ölçümü, ikinci yapay zekâ modeli, elle etiketleme) ve her bulgunun yanında bu teyidin sonucunu gösterir.
- **Önceliklendirme:** Aynı hatayı üreten şablonları ve CSS öğelerini bulur ("bu tek düzeltme 3.000 sayfayı düzeltir"). Böylece bulgular uzun bir liste olmaktan çıkıp yapılacak iş listesine dönüşür.
- **Düzeltme önerisi:** Mobil taşmalar için sitenin kendi sınıf adlarıyla CSS kuralı üretir, kuralı canlı sayfaya (yalnız tarayıcı sekmesine) ekleyip yeniden ölçer ve başka bir yeri bozup bozmadığını raporlar.
- **Teslim edilebilirlik:** Bütün raporları ve ham veriyi anlaşılır adlarla tek bir klasörde, içindekiler sayfasıyla toplar.

Bunlara ek olarak, aynı programın içinde bağımsız bir modül olarak **OİBS Bologna Bilgi Paketi**'ni tarayıp ilişkisel bir PostgreSQL veritabanına aktaran bir araç bulunur (birimler, programlar, ders planları, ders içerikleri ve öğrenme çıktısı × program çıktısı katkı matrisi).

1.3 Arka plan

Program, Mersin Üniversitesi Öğrenci İşleri Daire Başkanlığı (ÖİDB) web sitesi üzerine yapılan staj çalışmasından doğdu (`mehmetfatihvar/staj` reposu). Staj boyunca her iş için ayrı bir komut satırı aracı yazıldı: keşif, kanıtlı test, link ve belge denetimi, kırık sayfa doğrulama, şablon kümeleme, yapay zekâ

destekli görsel denetim, canlı doğrulama, CSS düzeltme ve Bologna aktarımı. Web Denetim Merkezi bu araçları **değiştirmeden** tek bir masaüstü programında birleştirir; araçları doğru sırayla, doğru parametrelerle çalıştırır, ilerlemeyi gösterir, sonuçları okunur raporlara dönüştürür ve programı siteye özgü olmaktan çıkarıp **site profilleri** ile her siteye uygulanabilir hâle getirir.

1.4 Temel kavramlar

Kavram	Anlamı
Site profili	Bir sitenin bütün ayarları: adres, ad, keşif kuralları, test parametreleri, yapay zekâ talimatındaki site tanımı. `profiller/<ad>.json` dosyasında durur.
Site haritası	Test edilecek sayfaların listesi (JSON, `url_list`). Keşif aracı üretir.
Adım	Programın çalıştırdığı tek iş (ör. "Tüm site kanıtlı testi"). Her adım bir ya da birkaç komut çalıştırır ve dosya üretir. Programda 39 adım vardır.
Hat	Sırayla çalışan adımlar zinciri. Arayüzde ana sayfadaki **Başlat** sıradaki aşamayı, **Kalanları da sırayla çalıştır** kalan aşamaları çalıştırır; komut satırında **tam hat** ve **yapay zekâ hattı** vardır.
Şablon	Aynı düzenden üretilen sayfalar (ör. bütün bölümlerin "program hedefleri" sayfası). Görsel hatalar çoğunlukla şablona bağlıdır.
Temsilci	Bir şablonu (ya da görsel kümeyi) temsilen ayrıntılı denetlenen sayfa.
Dilim	Tam sayfa ekran görüntüsünün yapay zekâya tek seferde gönderilen yatay parçası.
İlk model / hakem	Yapay zekâ denetiminde dilimlere önce bakan model (varsayılan Gemma) ve şüpheli dilimlere ikinci görüş veren model (varsayılan Gemini Flash-Lite).
Teyit / doğrulama	Bir bulgunun ikinci, bağımsız bir yolla yeniden sınanması.
Kanıt	Bulguyu destekleyen dosya: ekran görüntüsü, işaretli görüntü, CSV satırı, kayıt satırı.

2. Genel bakış

2.1 Program hangi soruları cevaplar?

Soru	Yöntem	Sonuç dosyası
Sitede hangi sayfalar var?	Başlangıç adresinden gerçek bağlantıları genişlik öncelikli izleme (keşif)	`kesif/site_haritasi.json`
Sayfalar açılıyor mu, hangileri kırık?	Her sayfa Chromium'da açılır, HTTP durumu kaydedilir	`test_output/report.html`, `results.jsonl`
Güvenlik başlıkları, Türkçe karakter, erişilebilirlik, mobil uyum, JS hataları nasıl?	Aynı ziyaret sırasında yanıt başlıkları ve DOM ölçülür	`report.html`, `report.xlsx`
Hangi bağlantılar kırık, hangi sayfada, hangi butonda?	Sayfalardaki bütün bağlantılar toplanır, tekilleştirilir, HTTP ile denenir	`linkdenetim/kirik_link_raporu.html`
Belgeler (PDF/Word/Excel) erişilebilir mi, yerel dosya yolu sızmış mı?	Ağ yanıtları ve DOM'daki belge bağlantıları	`denetim/belge_raporu.html`
Kırık görünen sayfa gerçekten kırık mı?	Doğrudan tarayıcıda açma + bağlantı veren sayfada butona gerçekten tıklama	`routes_verification.csv`, `clicks_verification.csv`, raporlardaki Teyit sütunu
Görünüm hataları var mı?	DOM ölçümü (şablon temsilcileri) + piksel taraması (bütün görüntüler) + yapay zekâ (temsilci dilimleri)	`gorsel/gorsel_rapor.html`, `visual_audit/denetim_cikti/rapor/rapor.html`
Bulunan görsel hata gerçek mi?	Canlı sitede DOM'da yeniden ölçme, elle etiketleme	`dogrulama_cikti/sonuc.csv`, `etiket_ozet.txt`
Taşma hangi CSS ile düzelir, düzeltme başka yeri bozar mı?	Öğe türüne göre CSS üretme, ekleyip yeniden ölçme	`oto_cikti/oneri.css`, `css_ozet.txt`
Bologna verisi veritabanına nasıl aktarılır?	OİBS sayfalarını tarayıp ilişkisel tablolara yazma (Go)	PostgreSQL

2.2 Tasarım ilkeleri

Programın bütün parçaları birkaç ortak ilkeye göre yazıldı. Bu ilkeler, ileride yapılacak değişikliklerde de korunmalıdır:

İlke	Ne demek, nerede uygulanıyor?
Siteyi değiştirmemek	Program yalnız okur. CSS denemeleri yalnız programın açtığı tarayıcı sekmesine eklenir (`--css`). Hız sınırı ve eşzamanlılık ayarlanabilir (varsayılan 4 sekme).
Her bulgunun kanıtı olsun	Her sayfanın ekran görüntüsü; görsel kusurlarda kırmızı kutulu işaretli görüntü; yapay zekâ bulgularında `evidence_text` (görünen metnin aynısı) zorunlu alan.
Doğrulanmamış bulgu kesin sayılmaz	Kırık link → doğrudan erişim + tıklama teyidi; yapay zekâ → iki model oyu (KESİN/İNCELE) + canlı DOM ölçümü + elle etiketleme; isabet oranları ölçülüp rapora yazılır.
Ucuz ve kesin olan önce	Kodla ölçülebilen her şey (taşma, boş sayfa, eksik eş, mojibake) yapay zekâya sorulmadan kodla ölçülür; yapay zekâ yalnız kodla ölçülemeyen görsel yargılar için kullanılır.

İlke	Ne demek, nerede uygulanıyor?
Kaldığı yerden devam	Uzun işlerin hepsi (keşif, test, link/belge, analiz, Bologna) yarıda kalırsa aynı komutla devam eder; yapılmış iş tekrarlanmaz (`results.jsonl`, `sonuclar.jsonl`, `tarama_ham.csv`, `scrape_runs`).
Araçlar değiştirilmeden çağrılır	`merkez/adimlar.py` araçlara yalnız komut satırı argümanı verir. Araçlar tek başına da çalışır; **sözleşme testleri** programın gönderdiği her seçeneğin araçta tanımlı olduğunu denetler.
Siteye özgü hiçbir şey kodda olmasın	Adres, ad, keşif kuralları, yerel ayar, User-Agent, yapay zekâ talimatındaki site tanımı ve dil profil dosyasındadır.
Her adımın görünür bir raporu olsun	Kendi raporu olmayan adımlar dahil her adımın sonunda `adim_raporlari/` altına HTML rapor yazılır.
Arayüz donmasın	İşler arka planda çalışır; arayüz olayları zaman bütçeli bir kuyrukla işlenir; durum hesapları ayrı iş parçacığında ve dosya imzası önbelleğiyle yapılır.

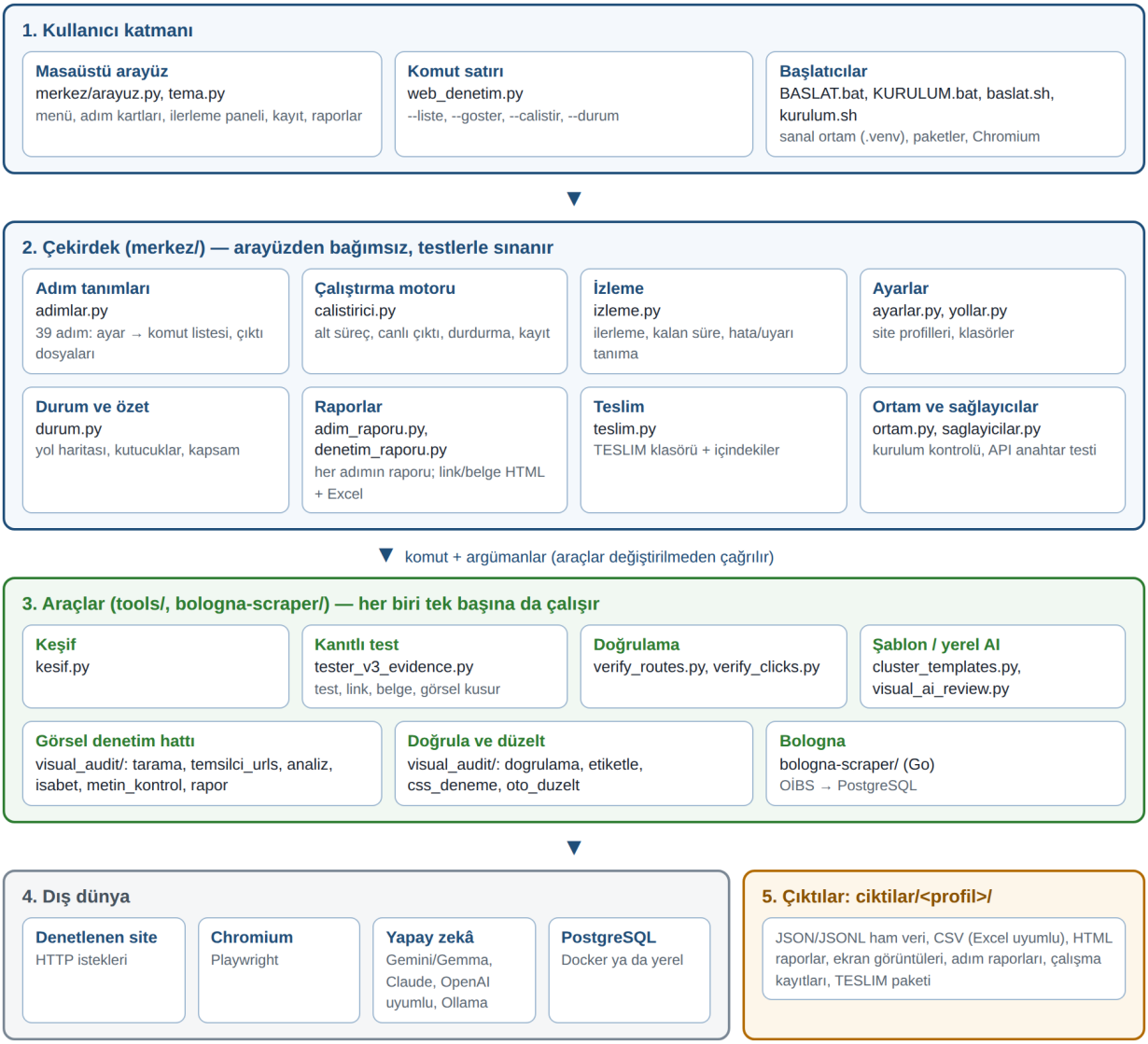
2.3 Kullanılan teknolojiler

Teknoloji	Nerede, neden
Python 3.9+	Programın ve araçların dili. Windows ve Linux'ta, 3.9 ve 3.12 ile test edilir.
Tkinter (+ isteğe bağlı sv-ttk)	Masaüstü arayüz. Python ile birlikte gelir, ek kurulum istemez. Varsayılan "sade" görünüm yerleşik clam teması üzerine çizilir (Windows'ta 5–7 kat daha hızlı).
Playwright + Chromium	Gerçek tarayıcıda sayfa açma, ekran görüntüsü, DOM ölçümü, tıklama. Kurulumda indirilir.
requests + BeautifulSoup	Hızlı (tarayıcısız) keşif.
Pillow	Görüntü ölçümü, dHash, dilimleme, kesit üretme.
pandas + openpyxl	Test sonuçlarının Excel raporu.
google-genai, anthropic, ollama, urllib	Yapay zekâ sağlayıcıları: Gemini/Gemma, Claude, yerel Ollama, OpenAI uyumlu servisler (OpenAI, OpenRouter, Groq, Mistral, LM Studio, vLLM …).
Go + PostgreSQL (+ Docker)	Bologna aktarım aracı. Docker ile Go/PostgreSQL kurmadan çalışır.
GitHub Actions	Her değişiklikte birim testleri (Windows + Linux, Python 3.9 + 3.12) ve Bologna testleri (Go, PostgreSQL servisiyle).
Node.js docx + LibreOffice	Yalnız dokümantasyon üretimi (bu doküman ve kullanım kılavuzu).

3. Mimari

3.1 Katmanlar

Program beş katmandan oluşur. Üst katman alttakini kullanır, alttaki üsttekini bilmez: araçlar programı bilmez, çekirdek arayüzü bilmez. Bu sayede araçlar tek başına komut satırından, çekirdek ise arayüz olmadan (komut satırı ve testlerden) çalışabilir.



Şekil 1. *Katmanlı mimari: kullanıcı katmanı, çekirdek, araçlar, dış dünya ve çıktılar*

Katman	İçerik	Sorumluluk
1. Kullanıcı	`merkez/arayuz.py`, `merkez/tema.py`, `web_denetim.py`, başlatıcılar	Kullanıcıdan komut almak, ilerlemeyi ve sonuçları göstermek, kurulum.
2. Çekirdek	`merkez/` altındaki diğer modüller	Adım tanımları, çalıştırma, ilerleme izleme, ayarlar, durum, raporlar, teslim. Arayüzden bağımsızdır.
3. Araçlar	`tools/`, `tools/visual_audit/`, `bologna-scraper/`	Asıl işi yapan programlar. Her biri tek başına çalışır, girdi/çıktısı dosyadır.

Katman	İçerik	Sorumluluk
4. Dış dünya	Denetlenen site, Chromium, yapay zekâ servisleri, PostgreSQL	Program bunlara yalnız araçlar üzerinden erişir (sağlayıcı testi hariç).
5. Çıktılar	`ciktilar/<profil>/`	Bütün ham veri ve raporlar. Her profilin çıktısı ayrı klasördedir.

3.2 Klasör yapısı

```
web-denetim-merkezi/
├─ web_denetim.py            program girişi (arayüz ya da komut satırı)
├─ BASLAT.bat / baslat.sh    başlatıcı (ilk açılışta kurulumu çalıştırır)
├─ KURULUM.bat / kurulum.sh  .venv sanal ortamı, paketler, Chromium
├─ requirements.txt          Python paketleri
├─ merkez/                   çekirdek + arayüz (14 modül)
├─ tools/                    araçlar: keşif, test, doğrulama, şablon, yerel AI
│  └─ visual_audit/          yapay zekâ görsel denetim hattı + doğrula ve düzelt
├─ bologna-scraper/          OİBS Bologna → PostgreSQL (Go)
├─ profiller/                site profilleri (meu-oidb.json hazır gelir)
├─ data/meu-oidb/            ÖİDB için hazır site haritası
├─ tests/test_merkez.py      birim, sözleşme ve arayüz testleri
├─ docs/                     kullanım kılavuzu, bu doküman ve kaynakları
├─ .github/workflows/        sürekli entegrasyon (CI)
├─ ayarlar.json              genel ayarlar ve anahtarlar (git dışı)
└─ ciktilar/<profil>/        bütün çıktılar (git dışı)
```

3.3 Bir adımın yaşam döngüsü

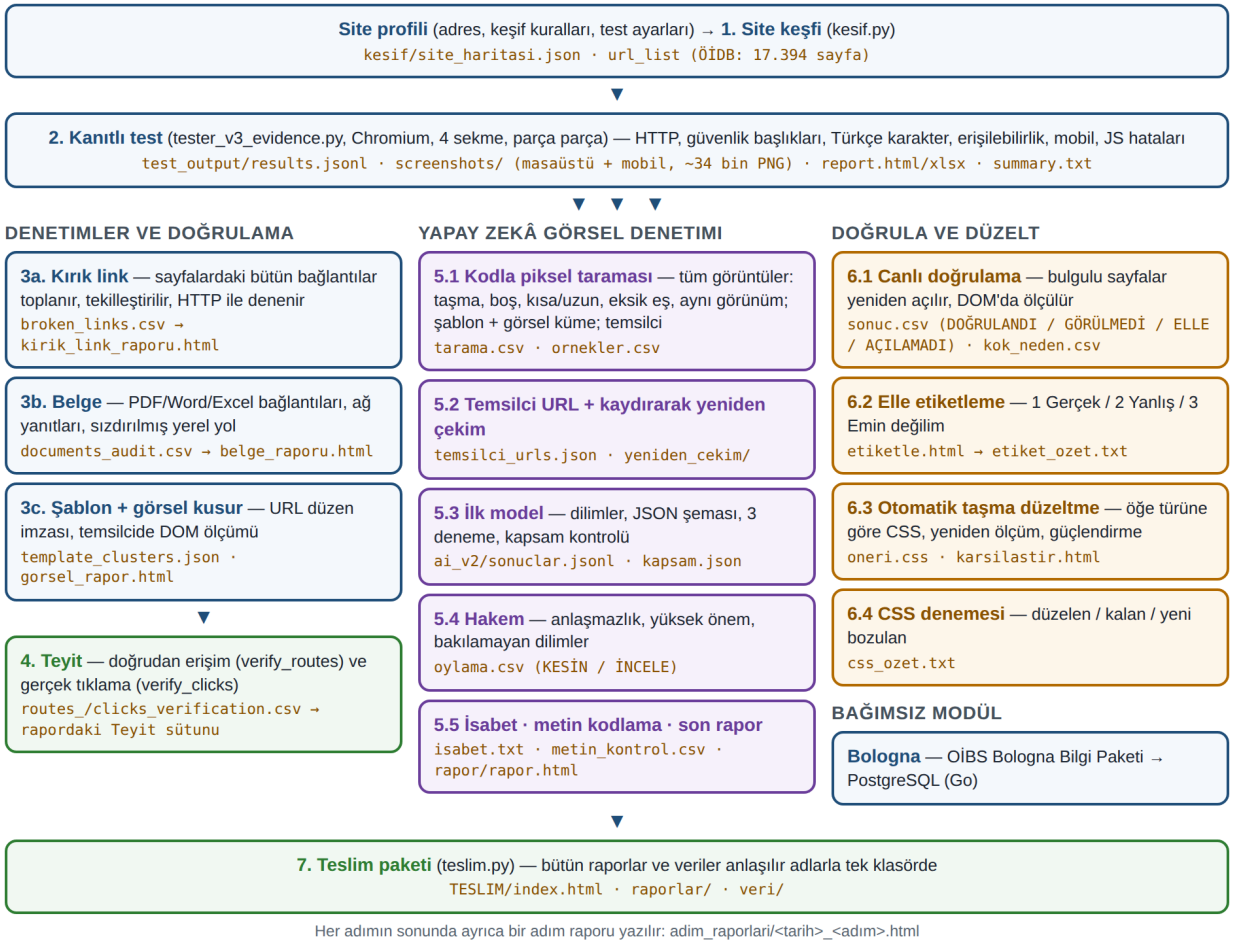
Kullanıcı bir adımı ya da hattı başlattığında şunlar olur (ayrıntı: 9. bölüm):

1. Arayüz (ya da komut satırı) seçilen adımları `Calistirici`'ya verir; iş arka plan iş parçacığında başlar.
2. Her adım için `Adim.komutlar(ayar)` çağrılır: etkin profilin ayarlarından araç komutları (argv, çalışma klasörü, ortam değişkenleri) üretilir. Örneğin tüm site testi, site haritasındaki sayfa sayısına göre parçalara bölünür (varsayılan 2.000 sayfalık parçalar).
3. Her komut alt süreç olarak çalışır; çıktısı canlı olarak okunur, kayıt dosyasına yazılır, satır satır `izleme.py`'ye verilir.
4. İzleme satırdan ilerlemeyi (`[946/2000]`) ve hata/uyarıyı tanır; kalan süreyi hesaplar; arayüze olay gönderir.
5. Komut sıfırdan farklı kodla biterse adım başarısız sayılır ve ("hata olursa sürdür" kapalıysa) hat durur.
6. Adım bitince adım raporu yazılır, son çalışma özeti güncellenir; durum ekranı çıktı dosyalarına bakarak adımı ✔️ ile işaretler.

4. Uçtan uca süreç

4.1 Süreç haritası

Aşağıdaki şekil verinin programdaki yolculuğunu gösterir. Her kutu bir adım ya da adım grubudur; altındaki kod yazısı ürettiği ana dosyadır. Okların yönü veri bağımlılığıdır: bir adım, yukarısındaki adımın çıktısını girdi olarak kullanır.



Şekil 2. *Uçtan uca süreç: keşiften teslim paketine kadar adımlar ve ürettikleri dosyalar*

Ana sayfadaki **yol haritası** bu sırayı izler: Site haritası → Deneme testi → Tüm site testi → Denetimler → Doğrulama → Yapay zekâ: hazırlık → Yapay zekâ: ilk model → Yapay zekâ: hakem ve rapor → Canlı doğrulama → Elle etiketleme: sayfa → Elle etiketleme: sonuç → Taşma düzeltme → Teslim paketi. Bologna bağımsızdır, herhangi bir zamanda çalıştırılabilir.

4.2 Hazırlık

Adım	Ne yapar	Kod
Bağımlılıkları kur	`requirements.txt` paketlerini ve Playwright Chromium'u kurar.	`adimlar._kurulum`
Ortam kontrolü	Paketler, Chromium, Go ve Docker kurulu mu; eksikleri ana sayfada uyarı olarak gösterir.	`merkez/ortam.py`

Adım	Ne yapar	Kod
Yapay zekâ sağlayıcılarını dene	Anahtarla her sağlayıcının model listesini çeker, ayardaki model adının listede olup olmadığını söyler. Ek paket gerektirmez (urllib).	`merkez/saglayicilar.py`

4.3 Site keşfi

Amaç: Test edilecek sayfaların listesini çıkarmak. **Araç:** `tools/kesif.py`.

Yöntem: Başlangıç adresinden itibaren sayfalardaki gerçek `<a href>`, `<area href>` bağlantılarını ve GET formlarını **genişlik öncelikli** izler. URL tahmin etmez; yalnız sitenin kendisinin gösterdiği bağlantıları izler. İki biçimi vardır:

- **İstek modu** (varsayılan): requests + BeautifulSoup. Hızlıdır; HTML'i sunucuda üreten siteler için yeterlidir.
- **Tarayıcı modu** (`--tarayici`): Playwright/Chromium. Bağlantıları JavaScript ile üreten (SPA, ör. React) siteler içindir; her işçinin kendi tarayıcısı olur.

Kurallar (`Kurallar` sınıfı) sonsuz kombinasyonu ve gereksiz yükü önler: aynı site sınırı, hariç desenler (ör. `/arama`), silinecek sorgu parametreleri (ör. `sort`), aynı yol için sorgu sınırı, saniyede istek sınırı, adres normalleştirme (parça `#…` atılır). Keşif kesilirse kuyruk ve görülenler kaydedilir; aynı komut kaldığı yerden devam eder.

Çıktı: `kesif/site_haritasi.json` (`url_list`). ÖİDB için hazır harita `data/meu-oidb/` altında gelir (17.394 sayfa). "Keşif haritasını kullan" adımı yeni haritayı profilin kaynağı yapar.

4.4 Kanıtlı test

Amaç: Haritadaki her sayfayı gerçek bir tarayıcıda açıp test etmek ve kanıtlamak. **Araç:** `tools/tester_v3_evidence.py` (kodun en büyük aracı, ~1.600 satır).

Her sayfa için `test_page` fonksiyonu tek bir tarayıcı ziyaretinde şunları ölçer:

Kontrol	Nasıl ölçülüyor
HTTP durumu, yüklenme süresi	Ana belge yanıtı; yönlendirme sonrası son adres.
Güvenlik başlıkları	Gerçek yanıt başlıklarında HSTS, CSP, X-Frame-Options, X-Content-Type-Options, X-XSS-Protection, Referrer-Policy var mı.
Türkçe karakter	Ekrana çizilmiş metinde (`innerText`) Türkçe harf sayıları; Ã, Å, Ä± gibi bozuk dizilerin sayısı 5'i geçerse mojibake şüphesi.
Erişilebilirlik	Alt metni olmayan görseller, H1 sayısı (yok / birden çok), etiketsiz form alanları, `lang` özniteliği, sayfa başlığı.
Formlar	Form sayısı, yöntem, alan sayısı.
Mobil uyum	viewport meta etiketi ve medya sorgusu (media query) var mı; 375 px'te görünüm.
JavaScript hataları	Konsol hataları sınıflandırılır: gerçek JS hatası ile kaynak yükleme hatası ayrılır; yalnız gerçek JS hatası uyarı sayılır.
Kanıt	Masaüstü (1366×900) ve mobil (375×812) tam sayfa ekran görüntüsü; isteğe bağlı tablet (768×1024) ve video.

Parça parça çalışma: Program tüm site testini `--offset/--limit` ile parçalara böler (varsayılan 2.000 sayfa). Her sayfanın sonucu tek satır olarak `results.jsonl`'e eklenir; yarıda kalan test yeniden çalıştırılınca yapılmış sayfaları atlar. Rapor her zaman dosyadaki **bütün** geçmişten üretilir (`report.html`, `report.xlsx`, `summary.txt`). "Hatalı sayfaları yeniden test et" yalnız hata/zaman aşımı alanları düşük eşzamanlılıkla yeniden dener.

> **Not**
> İlerleme yüzdesi ve kalan süre, sayfalar sırasız bittiği için liste sırasıyla değil **tamamlanan iş sayacıyla** basılır; programdaki yüzde geri gitmez ve kalan süre son 10 dakikanın hızına göre hesaplanır (9.4).

4.5 Belge ve kırık link denetimi

Bu iki denetim de test aracının ayrı modlarıdır (`--check-docs`, `--check-links`) ve ayrı çıktı klasörlerinde çalışır.

Kırık link denetimi

1. Her sayfa ziyaretinde sayfadaki bütün bağlantılar **metinleriyle ve bulundukları sayfayla** toplanır.
2. Bağlantılar tam adreslerine göre tekilleştirilir. Adresin `#…` parçası atılmaz: aynı sayfanın farklı `#` parçaları ayrı bağlantı sayılır ve ayrı denenir (sunucuya giden istek aynı olduğu için sonuçları da aynıdır).
3. Her tekil bağlantı bir kez HTTP ile denenir (`check_broken_links`). 400 ve üstü durum ya da bağlantı hatası **kırık** sayılır.
4. Sonuç `broken_links.csv`'ye yazılır: adres, durum, bağlantıyı veren sayfa, buton metni.
5. Program bu CSV'den süzülebilir bir HTML rapor (`kirik_link_raporu.html`) ve Excel'de doğru açılan bir CSV üretir (`merkez/denetim_raporu.py`). Durumlar okunur sınıflara çevrilir: 404 Bulunamadı, 410 Kaldırılmış, 403 Erişim yasak, 401 Giriş gerekli, Sunucu hatası (5xx), DNS, zaman aşımı, SSL, bağlantı reddedildi.

ÖİDB'de 17.394 sayfada 36.899 tekil bağlantı bulundu. Sayfa sayısından fazla olmasının nedeni, sayfaların site dışına (sosyal medya, diğer birimler, belgeler) ve haritada olmayan adreslere de bağlantı vermesidir.

Belge denetimi

Belgeler iki kaynaktan toplanır: sayfa yüklenirken **ağ yanıtları** (belge türünde içerik) ve DOM'daki belge bağlantıları (metin ve kaynak sayfayla). `audit_documents` durumu bilinmeyenleri HTTP ile dener ve her belgeye bir bayrak verir:

Bayrak	Anlamı
erisilebilir	Belge açılıyor.
KIRIK/ERISILEMIYOR	400 ve üstü durum.
ULASILAMADI	Bağlantı hatası (DNS, zaman aşımı …).
SIZDIRILMIS YEREL DOSYA YOLU	Bağlantı `file:` ile başlıyor: içerik girilirken bilgisayardaki dosya yolu yapıştırılmış. Hem kırıktır hem de bilgi sızıntısıdır.

Sonuç `documents_audit.csv` → `belge_raporu.html` (varsayılan görünüm yalnız sorunlu belgeler) + Excel CSV.

4.6 Şablon kümeleme ve görsel/UI kusur denetimi

Şablon kümeleme (`tools/cluster_templates.py`): URL'ler **düzen imzasına** indirgenir (dil sonrası ilk ve son yol parçası, sayısal kimlikler genelleştirilir). Aynı imzalı sayfalar aynı şablondur; ÖİDB'de 17 bin sayfa yaklaşık 600 şablona iner. Her şablondan birkaç temsilci seçilir. Sonuç: `template_clusters.json`, `template_representatives.json`.

Görsel/UI kusur denetimi (test aracının `--check-visual` modu, şablon temsilcilerinde): Tarayıcı içinde çalışan bir JavaScript (`VISUAL_AUDIT_JS`) sayfayı ölçer. Yaklaşımı bilinçli olarak tutucudur; piksel eşiği kovalamaz, yalnız "kullanıcının gerçekten gördüğü" bozukluğu bildirir:

- **İçerik taşması:** ekranda fiilen çizilen, metin ya da görsel içeren, kırpılmayan bir öğe ekranın sağına taşıyor ve sayfa yana kayıyorsa. Boş/dekoratif kutular ve gizli paneller elenir.
- **Kırık görsel**, **kırpılan metin**, **üst üste binen öğeler**, **boş render**.

Her kusurlu bölge kırmızı kutuyla işaretlenip ayrı bir kanıt görüntüsü alınır. Rapor (`gorsel_rapor.html`) kusurları şablona bağlar ve en çok sayfayı etkileyen şablonları öne alır.

4.7 Kırık sayfa doğrulama (teyit)

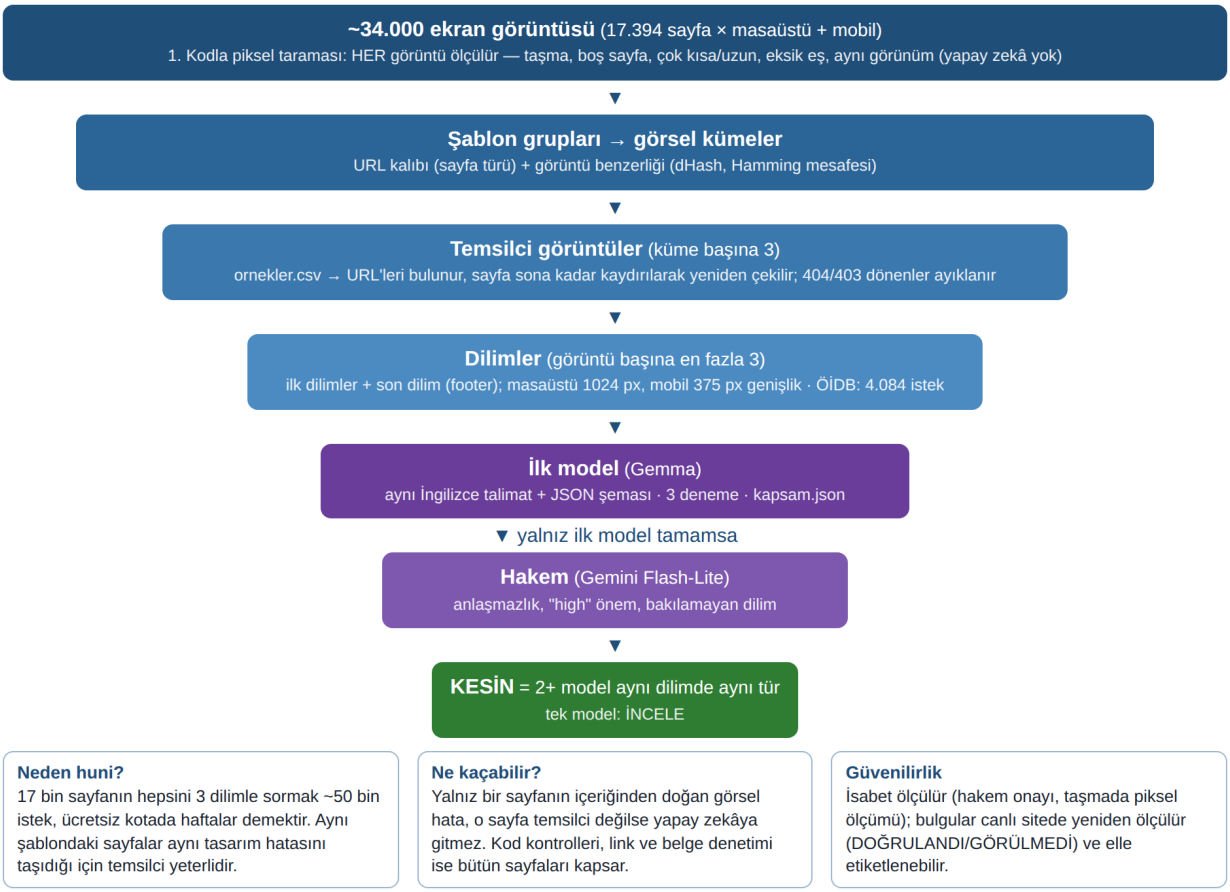
Otomatik HTTP kontrolü bazı sitelerde yanıltıcıdır. Örneğin React tabanlı bir sitede `/programlar` sekmesi tıklanınca çalışır (istemci tarafı yönlendirme), ama adrese doğrudan gidilince sunucu 403/404 döner. Bu bir kırık sayfa değil, "derin bağlantı" sorunudur. Bunu ayırmak için iki bağımsız teyit yapılır:

Teyit	Yöntem	Kararlar
Doğrudan erişim `tools/verify_routes.py`	401/403/404 dönen her adres tarayıcıda açılır, React'in çizmesi beklenir; yeterli metin (300+ karakter), ana içerik öğesi var ve "yasak/bulunamadı" yazmıyorsa içerik geliyor sayılır.	ÇALIŞIYOR (200) · İÇERİK GELİYOR (durum ≠ 200 ama tarayıcıda açılıyor) · GERÇEKTEN KIRIK
Gerçek tıklama `tools/verify_clicks.py`	Kırık adrese bağlantı veren çalışan bir sayfa bulunur, tarayıcıda açılır, hedefe giden <a> öğesi DOM'da bulunup **gerçekten tıklanır**; tıklama sonrası nereye gidildiği ve içerik gelip gelmediği ölçülür.	KIRIĞA GÖTÜRÜYOR · ÇALIŞAN YERE GİDİYOR (yanlış alarm) · LİNK/BUTON YOK · REFERRER AÇILMADI

Teyit sonuçları kırık link raporunda her satırın yanındaki **Teyit** sütununda gösterilir ve bu sütuna göre süzülebilir. Böylece rapordaki "kırık" sayısı, gerçekten kullanıcıyı kırık sayfaya götüren bağlantılarla yanlış alarmlara ayrılır.

4.8 Yapay zekâ destekli görsel denetim

Kodla ölçülemeyen görsel hatalar (üst üste binme, eksik veri, bozuk ikon, okunmaz renk, bozuk menü …) için ekran görüntüleri görüntü anlayabilen yapay zekâ modellerine sorulur. Bu hat, maliyeti ve süreyi makul tutmak için bir **huni** gibi çalışır:



Şekil 3. *Yapay zekâ görsel denetiminin hunisi: 34 bin görüntüden kesin bulguya*

1. Kodla piksel taraması — tools/visual_audit/tarama.py

Bütün ekran görüntüleri (yapay zekâsız, paralel) ölçülür: boyut, gri ton standart sapması, iki boyutta dHash (görüntü parmak izi). Bayraklar:

Bayrak	Kural
TASMA(+N px)	Görüntü genişliği o cihazın normal genişliğinden büyük: sayfa yana kayıyor (kesin ölçüm).
DAR(−N px)	Normalden dar.
BOS_SAYFA	Gri ton standart sapması < 4: sayfa boş ya da tek renk.
COK_KISA / COK_UZUN	Yükseklik medyanın %30'undan kısa ya da 5 katından uzun.
EKSIK_ES	Sayfanın masaüstü ya da mobil eşi yok.
AYNI_GORUNUM(n)	En az 3 farklı adres birebir aynı görünüyor (soft-404, yönlendirme, boş içerik şüphesi).
BOZUK_DOSYA	Görüntü açılamadı.

Ardından sayfalar URL kalıbına göre **şablon gruplarına**, grup içinde görsel benzerliğe göre (dHash Hamming mesafesi eşik altında) **görsel kümelere** ayrılır; her kümeden 3 **temsilci** seçilir (`ornekler.csv`).

2. Temsilci URL ve kaydırarak yeniden çekim

`temsilci_urls.py` görüntü dosya adlarını (URL'den kararlı biçimde üretilmiş: yol + md5) site haritasıyla eşleştirip temsilcilerin adres listesini çıkarır. Temsilciler sayfa sonuna kadar **kaydırılarak** yeniden çekilir: tembel yüklenen görseller ve sayaçlar ilk çekimde boş görünüp yanlış alarm üretmesin diye. HTTP 200 dönmeyen sayfalar analizden ayıklanır.

3. İlk model — analiz.py

Her temsilci görüntü hedef genişliğe küçültülür (masaüstü 1024 px, mobil 375 px) ve dilimlere bölünür; uzun sayfalarda ilk iki dilim ve footer'ı yakalamak için son dilim alınır (en fazla 3). Her dilim **aynı İngilizce talimat ve aynı JSON şemasıyla** modele gönderilir. Talimat modelden yalnız gerçek bir ziyaretçinin fark edeceği görsel hataları, **görünen metni kanıt olarak birebir kopyalayarak** istemesini ister. Şemadaki hata türleri:

`horizontal_overflow`, `overlap`, `cut_off_text`, `broken_image`, `misalignment`, `low_contrast`, `mojibake`, `empty_gap`, `broken_navigation`, `missing_data`, `other`; her biri için önem (low/medium/high), güven, konum, kanıt metni ve açıklama.

Dayanıklılık önlemleri: dakikalık hız sınırı (`HizSiniri`), geçici hatalarda artan beklemeyle 6 deneme (429/503/500), 5 dakika bağlantı zaman aşımı, Gemma'nın bazı PNG'lerde verdiği kalıcı 500 hatasına karşı JPEG ile ikinci deneme, günlük kota bitince modeli durdurma. Yapılamayan istekler aynı çalıştırmada **2 tur daha** denenir. Her cevap `sonuclar.jsonl`'e satır olarak yazılır; yalnız başarılı cevaplar "yapıldı" sayılır, bu yüzden yeniden çalıştırma yalnız eksikleri gönderir.

Bitişte **kapsam** hesaplanır (`kapsam.json`): beklenen dilim, tamamlanan, yapılamayan, hiç denenmeyen. Hiç denenmemiş ya da kota yüzünden kalan dilim varsa adım başarısız biter ve hat hakeme geçmez.

4. Hakem

Hakem başlamadan önce ilk modelin her dilimi denediğini doğrular. Sonra yalnız şu dilimleri ikinci bir modele sorar (`hakem_dilimleri`): bir hata türünü tek modelin bulduğu (anlaşmazlık), herhangi bir modelin "high" önem verdiği ve ilk modelin denemelere rağmen hiç bakamadığı dilimler. Ücretsiz kotalarda hakem yükü örneklemeyle azaltılabilir (şablon ya da hata türü başına en fazla N dilim).

Oylama (`birlestir`): aynı dilimde aynı hata türünü 2 ya da daha fazla model bulduysa **KESİN**, tek model bulduysa **İNCELE**. Sonuç `oylama.csv`, `bulgular.csv`, `ozet.txt` (model başına bulgu ve token kullanımı).

5. İsabet, metin kodlama ve son rapor

- **İsabet** (`isabet.py`): hata türü başına hakemin ilk modeli onaylama oranı; taşmada kodla ölçülmüş gerçek taşma (TASMA bayrağı) referans alınarak modelin isabeti (precision) ve yakalama oranı (recall).
- **Metin kodlama** (`metin_kontrol.py`, yapay zekâsız): görünen metinde `?` ile kaybolan Türkçe harf (Atat?rk, ?ZT?RK), ekrana basılan HTML kodu (`'`), mojibake (Ã¼), yer tutucu `&#65533;`. Bu kontrol, yapay zekânın İngilizce sayfalarda bulduğu bir hata sınıfının kodla bütün temsilcilerde aranması için eklendi.
- **Son rapor** (`rapor.py`): kodla kesinleşen bulgular, metin hataları, yapay zekâ bulguları (isabeti yeterli türlerde KESİN olanlar ekran görüntüsü kesitleriyle; tek modelliler "inceleme önerilir"; isabeti düşük türler yalnız sayı), isabet tablosu, yöntem ve sınırlar. Güvenilir sayılan türler: mojibake, overlap, missing_data, broken_image.

İsteğe bağlı bir alternatif olarak **yerel model incelemesi** (`tools/visual_ai_review.py`) şablon temsilcilerini bilgisayarda çalışan bir Ollama modeline sorar: anahtar, ücret ve internet gerekmez, veri bilgisayardan çıkmaz.

4.9 Doğrula ve düzelt

Adım	Ne yapar	Kod / çıktı
7. Canlı doğrulama	Kod, metin ve KESİN yapay zekâ bulgusu olan sayfalar yeniden açılır, DOM'da ölçülür. Karar: **DOĞRULANDI** (aynı hata aynı cihazda, yapay zekâda aynı dikey bölgede yeniden ölçüldü), **GÖRÜLMEDİ** (düzeltilmiş ya da çekim kaynaklı; kendi kutusunda kayan tablo da buraya düşer), **ELLE** (DOM ile ölçülemeyen tür), **AÇILAMADI**. Taşmalarda hatayı üreten öğenin CSS seçicisi bulunur; aynı seçici kaç sayfada çıkıyor sayılır.	`dogrulama.py` → `sonuc.csv`, `kok_neden.csv`, `ozet.txt`
8a/8b. Elle etiketleme	ELLE, GÖRÜLMEDİ ve DOĞRULANDI'lardan bir örneklem tek sayfada eski çekim ile şimdiki hal yan yana gösterilir; klavyeyle 1 Gerçek / 2 Yanlış / 3 Emin değilim. Etiketlerden tür başına elle ölçülmüş isabet ve otomatik doğrulamanın isabeti hesaplanır.	`etiketle.py` → `etiketle.html`, `etiket_ozet.txt`
10. Otomatik taşma düzeltme	Sayfa CSS'siz ölçülür; sayfayı yana kaydıran öğe gizle-ölç yöntemiyle bulunur ve sınıflandırılır: uzun metin (`overflow-wrap: anywhere`), tablo (`display:block; overflow-x:auto`), önbiçimli metin, gömülü içerik ve sabit genişlikli kutu (`max-width:100%`). Sitenin kendi sınıflarıyla kural yazılır, eklenip yeniden ölçülür; hâlâ kayanlar için yeni kural, etkisiz kural için `!important`. Son turda bütün sayfalar ölçülür; CSS'siz düzgün olup CSS'li bozulan ("yeni bozulan") sayfa varsa raporlanır.	`oto_duzelt.py` → `oneri.css`, `ozet.txt`, `karsilastir.html`
9. CSS denemesi	Verilen CSS'i canlı doğrulamada yana kayan sayfalara ekleyip ölçer: düzelen, kalan, yeni bozulan.	`css_deneme.py` → `css_ozet.txt`, `css_sonuc.csv`

4.10 Bologna aktarımı

OİBS kullanan üniversitelerin **Bologna Bilgi Paketi**'ni tarayıp ilişkileri koruyarak PostgreSQL'e yazan Go aracıdır (`bologna-scraper/`). Taranan veri: birimler, programlar (önlisans, lisans, yüksek lisans, doktora), program çıktıları, ders planları (yarıyıl, zorunlu/seçmeli, AKTS, seçmeli grup → havuz dersi), ders detayları (amaç, içerik, öğrenme çıktıları, haftalık konular, kaynaklar, değerlendirme, iş yükü) ve **öğrenme çıktısı × program çıktısı katkı matrisi**. Varsayılan site Mersin Üniversitesi'dir; başka bir site `--base-url` (arayüzde profilin **OİBS Bologna adresi**) ile taranır ve her site aynı veritabanında **kendi PostgreSQL şemasına** yazılır (program ve ders kimlikleri siteler arasında çakıştığı için). MEÜ geriye uyum için `public` şemasındadır, diğerlerinin şeması adresten türetilir (`obs.ornek.edu.tr` → `ornek`); `site_info` tablosu şemayı ilk taranan siteye bağlar ve başka bir sitenin aynı şemaya yazılmasını engeller. Mersin Üniversitesi'nin tamamı (4 seviye, 442 program, eski planlarla 95.432 ders) yaklaşık 3,5 saatte, 0 hatayla tarandı.

Sitenin özelliklerinden doğan tasarım kararları: tablolar ID ile değil önlerindeki başlık metniyle tanınır; ders bağlantıları `onclick` içindedir; eski yıllar ASP.NET form gönderimiyle (ViewState dahil) alınır ve her program kendi oturumunda istenir; ortak seçmeli dersler her programa ayrı kimlikle kopyalandığı için her ders kodu bir kez indirilip kopyalara yazılır (istek sayısı ~30 binden ~4.600'e iner); karakter kodlaması gerekirse windows-1254 kabul edilir; varsayılan 2 istek/sn ve 503/429'da geri çekilme.

4.11 Teslim paketi

`merkez/teslim.py` bütün çıktıları anlaşılır adlarla `TESLIM/` klasöründe toplar: `raporlar/` (test, kırık link, belge, görsel, yerel yapay zekâ, görsel denetim son raporu, CSS karşılaştırması …), `veri/` (Excel test sonuçları, ham sonuçlar, CSV'ler, site haritası, şablon kümeleri, yapay zekâ bulguları, doğrulama, etiket, CSS özetleri) ve bunların açıklamalı içindekiler sayfası `index.html`. Paketlemeden önce link ve belge raporları eskimişse yeniden üretilir.

5. Sonuçlar nasıl elde ediliyor?

5.1 Sonuç → süreç → dosya

Aşağıdaki tablo, raporlarda ve ana sayfada görülen her sayının hangi süreçten geçerek üretildiğini ve nasıl yorumlanması gerektiğini özetler.

Sonuç	Nasıl elde ediliyor	Dosya	Yorum / güven
Site haritası (sayfa sayısı)	Keşif: gerçek bağlantılar genişlik öncelikli	`site_haritasi.json`	Bağlantı verilmeyen (yetim) sayfalar bulunamaz; kurallar dışarıda bıraktıkları kadar eksiktir.
Test edilen sayfa, kapsam %	Tester'ın `results.jsonl` satır sayısı / harita	`results.jsonl`	Kesin sayım.
Kırık sayfa	Ana belge HTTP durumu ≥ 400	`results.jsonl`, `report.html`	SPA sitelerde yanıltıcı olabilir → rota teyidi.
Kırık bağlantı	Tekil bağlantıların HTTP denemesi	`broken_links.csv`	Teyit sütunu yanlış alarmları ayırır; dış sitelerin anlık hataları olabilir.
Sorunlu belge	Belge bayrağı ≠ erisilebilir	`documents_audit.csv`	Yerel dosya yolu bulgusu kesindir.
Görsel kusur	Şablon temsilcilerinde DOM ölçümü	`gorsel_denetim.csv`	Tutucu: görünür bozukluk şartı; şablon başına.
Piksel bayrakları	Bütün görüntülerin kodla ölçümü	`tarama.csv`	TASMA kesin ölçümdür; AYNI_GORUNUM şüphedir.
Yapay zekâ bulgusu (KESİN)	2+ model aynı dilimde aynı tür	`oylama.csv`	Tür başına isabet `isabet.txt`'te; canlı doğrulamaya gider.
İsabet oranı	Hakem onayı / piksel ölçümüyle karşılaştırma	`isabet.txt`	Hakem onayı alt sınırdır: iki model aynı hatayı yapabilir.
Metin kodlama bulgusu	Görünen metinde düzenli ifade kalıpları	`metin_kontrol.csv`	Kesin eşleşme; adres içindeki ? elenir.
DOĞRULANDI / GÖRÜLMEDİ	Canlı DOM ölçümü	`sonuc.csv`	Bulgunun bugün de var olup olmadığı.
Kök neden	Taşmayı üreten seçicinin sayfa sayısı	`kok_neden.csv`	Üst satırlar "tek düzeltme, çok sayfa"dır.
Düzelen / yeni bozulan	CSS'li ve CSS'siz ölçüm karşılaştırması	`css_ozet.txt`, `ozet.txt`	Yalnız yana kaymayı ölçer; görüntülere gözle bakılmalı.
Bologna kayıt sayıları	Veritabanı tablo sayımları	PostgreSQL	`scrape_errors` tablosu hataları tutar.

5.2 Güven katmanları

Program bir bulguyu birden fazla katmandan geçirerek kesinleştirir. Bir bulgu ne kadar çok katmandan geçtiyse o kadar güvenilirdir:

Bulgu	1. katman	2. katman	3. katman
Kırık bağlantı	HTTP durumu	Doğrudan tarayıcıda açma	Bağlantıya gerçekten tıklama

Bulgu	1. katman	2. katman	3. katman
Yatay taşma	Görüntü genişliği (piksel)	DOM'da sayfa gerçekten kayıyor mu + öğe	CSS ile düzeliyor mu, başka yer bozuluyor mu
Yapay zekâ görsel bulgusu	İlk model	Hakem (KESİN = 2+ model)	Canlı DOM ölçümü + elle etiketleme
Metin bozulması	Test aracının mojibake sayımı	Metin kodlama kalıpları (bütün temsilciler)	Canlı sayfada yeniden ölçüm

5.3 Sayılara dair örnek: ÖİDB

Ölçü	Değer
Site haritasındaki sayfa	17.394
Tekil bağlantı (link denetimi)	36.899
Ekran görüntüsü (masaüstü + mobil)	~34.000
Şablon (URL düzen imzası)	~600
Yapay zekâya giden istek (temsilci dilimleri)	4.084
Bologna: program / ders	442 / 95.432

> **Not**
> Değerler programın ÖİDB üzerindeki çalışmasından alınmıştır; site güncellendikçe değişir.

6. Kod haritası: çekirdek (merkez/)

Çekirdek 14 Python modülünden oluşur. Arayüz dışındaki modüller Tkinter'e bağımlı değildir ve doğrudan test edilir.

Dosya	Rol	Ana parçalar
web_denetim.py	Program girişi	Argüman yoksa arayüzü açar. `--liste` adımları, `--goster <adım>` çalışacak komutları gösterir (çalıştırmaz), `--calistir <adım|tam-hat|ai-hatti>` komut satırından çalıştırır, `--durum` çıktı durumunu yazar.
merkez/yollar.py	Sabit klasörler	Repo kökü, araç klasörleri, profil ve ayar dosyası yolları. Repo nereye kopyalanırsa kopyalansın yollar ona göre çözülür.
merkez/ayarlar.py	Ayarlar ve profiller	`PROFIL_VARSAYILAN` (siteye özgü) ve `GENEL_VARSAYILAN` (sağlayıcı, model, veritabanı) sözlükleri; `yukle`/`kaydet` iki dosyayı tek sözlükte birleştirip ayırır; `yeni_profil`; `etkin_ortam` (alt süreçlere API anahtarları); `Klasorler` (etkin profilin her aracının çıktı klasörü). Bilinmeyen anahtar ve yanlış tür sessizce yok sayılır.
merkez/adimlar.py	Adım tanımları	`Komut` (dış süreç ya da Python fonksiyonu) ve `Adim` (id, grup, ad, açıklama, komut üreteci, çıktı listesi, tahmini süre) veri sınıfları; 39 adımlık `ADIMLAR` listesi; komut üreticileri (`_tester`, `_tam_test`, `_analiz`, `_hakem`, `_dogrulama`, `_bologna` …); `tam_hat`, `AI_HATTI`; `onbellekli` (dosya imzası önbelleği), `harita_url_sayisi`.
merkez/calistirici.py	Çalıştırma motoru	`Calistirici.calistir(adimlar, ayar)`: adımları sırayla çalıştırır, alt süreç çıktısını canlı okur, kayıt dosyasına yazar, olay üretir; `durdur` (önce nazik sinyal, sonra süreç ağacını sonlandırma); her adım sonunda adım raporu.
merkez/izleme.py	Çıktı ayrıştırma	`ilerleme_oku` (4 ilerleme biçimi), `sorun_turu` (hata/uyarı tanıma), `SatirBolucu` (\n ve \r), `SureTahmini` (kayan pencereli kalan süre), `sure_metni`.
merkez/durum.py	Durum ve özet	`cikti_durumu` (adımın ana çıktısı var mı, ne zaman), `ozet` (Raporlar sayfasındaki sayılar), `kutucuklar` (ana sayfa), `yol_haritasi` (önerilen sıra; ilk model/hakem kapsamı `kapsam.json`'dan), CSV/JSONL okuyucular (önbellekli).
merkez/adim_raporu.py	Adım raporları	`uret`: her adımın sonucu, süresi, komutları, hataları, ürettiği dosyalar (CSV aranabilir tablo, TXT metin, JSON özet, HTML bağlantı) ve kayıt sonu; `dizin_yaz` (index.html), `son_rapor`.
merkez/denetim_raporu.py	Link ve belge raporları	`link_raporu` (Teyit sütunuyla), `belge_raporu`, Excel uyumlu CSV (BOM'lu, `;`), `link_turu`/`belge_turu` sınıflandırması; `_guncel` (CSV değiştiyse yeniden üret).
merkez/teslim.py	Teslim paketi	`paketle`: kaynak → hedef → açıklama listesinden TESLIM klasörü ve `index.html`.
merkez/ortam.py	Ortam kontrolü	`kontrol`: Python sürümü, eksik paketler, Chromium, Go, Docker; `sorunlar` (kullanıcıya gösterilecek eksikler).
merkez/saglayicilar.py	Sağlayıcı testi	Anthropic, Gemini, OpenAI uyumlu ve Ollama model listeleme uçlarını çağırır; ayardaki modelin varlığını doğrular.
merkez/tema.py	Tema	Açık/karanlık palet; "sade" görünümde clam temasını boyar, "modern" görünümde sv-ttk; Windows başlık çubuğu rengi.

Dosya	Rol	Ana parçalar
merkez/arayuz.py	Masaüstü arayüz	`Uygulama` sınıfı: sol menü, sayfalar, ana sayfadaki sıradaki adım kartı (tek **Başlat**), adım kartları (Çalıştır / Sonucu aç / Diğer), her zaman görünen durum şeridi (dönen gösterge, son hareket süresi ve 2/10 dakikalık sessizlik uyarıları, ilerleme, Durdur), açılır kapanır ayrıntılar paneli (kayıt/hatalar/son çalışma), ayar formu. Performans için: `_CizimiDondur` (Windows'ta çizimi geçici durdurma), sayfaların üst üste yığılıp öne alınması, durumun arka planda hesaplanması, zaman bütçeli olay kuyruğu.

6.1 Bir adımın tanımı

Bütün adımlar `merkez/adimlar.py`'deki tek listededir. Yeni bir araç programa bu listeye bir `Adim` eklenerek bağlanır; arayüz, komut satırı, durum ekranı, kayıtlar ve adım raporu yeni adımı kendiliğinden kullanır. Örnek:

```
Adim("ai_analiz", "ai", "3b. Yapay zekâ analizi",
     "İlk model aşaması. Görüntüleri dilimleyip seçili modellere ... sorar ...",
     lambda a: [_analiz(a, "--modeller", a["ai_modeller"])],       # komut üreteci
     lambda a: [K(a).ai_sonuc / "bulgular.csv",                      # ana çıktılar
                K(a).ai_sonuc / "oylama.csv", K(a).ai_sonuc / "ozet.txt"]),
```

`a` etkin ayar sözlüğü, `K(a)` o profilin `Klasorler` nesnesidir. Ana çıktıların hepsi varsa adım arayüzde "tamam" görünür; çıktının değişme zamanı adımın son çalışma zamanıdır.

7. Kod haritası: araçlar (tools/)

Araçların hepsi tek başına komut satırından çalışır (`python <araç> --help`). Program onları `merkez/adimlar.py` üzerinden doğru argümanlarla çağırır.

7.1 Keşif, test ve doğrulama araçları

Araç	Girdi → çıktı	Ana parçalar
`tools/kesif.py` (~470 satır)	başlangıç adresi → `site_haritasi.json`	`Kurallar` (normalleştirme, hariç desenler, sorgu sınırı), `IstekIndirici` (requests), `TarayiciIndirici` (Playwright), `Kesif` (genişlik öncelikli kuyruk, devam), `html_linkleri`.
`tools/tester_v3_evidence.py` (~1.600 satır)	site haritası → `results.jsonl`, `screenshots/`, `report.html/xlsx`, `summary.txt`; link/belge/görsel modlarında CSV'ler	`test_page` (tek sayfa), `check_broken_links`, `audit_documents`, `VISUAL_AUDIT_JS` + işaretleme, `build_html_report`, `build_visual_report`, `template_signature`, `safe_name` (URL → kararlı dosya adı), `run` (eşzamanlılık, parça, devam).
`tools/verify_routes.py`	test sonuçları → `routes_verification.csv`	`load_suspect_urls` (401/403/404), tarayıcıda açma, içerik ölçümü, karar.
`tools/verify_clicks.py`	`broken_links.csv` (ya da eski harita) → `clicks_verification.csv`	`load_broken_csv`, `load_from_map`, bağlantı veren sayfa seçimi, öğe bulma ve tıklama, karar.
`tools/cluster_templates.py`	site haritası → `template_clusters.json`, `template_representatives.json`	düzen imzası, küme sayımı, temsilci seçimi.
`tools/visual_ai_review.py`	temsilci görüntüler → `gorsel_ai.csv`, `gorsel_ai_rapor.html`	`ollama_tags`, `prep_image` (genişliğe göre küçültme), `ollama_review` (yapılandırılmış cevap), `build_ai_report`.

7.2 Görsel denetim hattı (tools/visual_audit/)

Araç	Aşama	Ana parçalar
`tarama.py`	1. Piksel taraması, şablon ve küme, temsilci	`olc` (paralel ölçüm), `dhash`, `sablon_anahtari`, `gorsel_kumeler` (açgözlü kümeleme), `analiz` (bayraklar), `ciktilari_yaz`. Ham ölçüm önbelleği `tarama_ham.csv`: yarıda kalırsa ölçülenler atlanır.
`temsilci_urls.py`	2a. Temsilci adresleri	`safe_name` (tester ile birebir aynı), `harita_urlleri`, eşleşmeyenler listesi.
`analiz.py` (~1.300 satır)	3. İlk model, hakem, oylama	İstemciler: `Claude`, `Gemini`, `Ollama`, `OpenAIUyumlu`; `HizSiniri`; `dilimle/dilim_numaralari`; `tek_istek` (deneme ve bekleme); `calistir/turlarla_calistir`; `kapsam`, `ilk_model_tamam_mi`; `hakem_dilimleri`; `toplu_calistir` (Claude Batch API, %50 ucuz); `birlestir` (oylama); `tahmin` (maliyet/süre); `gemini_dene` (çalışan modeli bulma).

Araç	Aşama	Ana parçalar
`isabet.py`	4. İsabet	hakem onay oranı, taşmada precision/recall, mojibake kanıt metinleri.
`metin_kontrol.py`	5. Metin kodlama	kalıplar: soru işareti, HTML kodu, mojibake, yer tutucu; `kelime` (adres içindeki eşleşmeleri eleme), `baglam`.
`rapor.py`	6. Son rapor	`ai_kayitlari`, `kesit` (dilim görüntüsü kesiti), güvenilir türler, isabet ve yöntem bölümleri.
`dogrulama.py`	7. Canlı doğrulama	`hazirla` (bulgu kaynakları: tarama, KESİN yapay zekâ, metin, HTTP), `kesin_ai_bulgulari`, `karsilastir` (karar, kök neden).
`etiketle.py`	8. Elle etiketleme	`olustur` (tek sayfa HTML, eski/yeni kesitler), `ozet` (elle ölçülmüş isabet).
`css_deneme.py`	9. CSS denemesi	`hazirla`, `karsilastir` (düzelen, kalan, yeni bozulan).
`oto_duzelt.py`	10. Otomatik düzeltme	tarayıcı içi bulucu (gizle-ölç), `kural_govdesi` (tür → CSS), `Kurallar` (birleştirme, `!important` ile güçlendirme), `orneklem` (şablon çeşitliliği), turlar.

8. Kod haritası: Bologna (bologna-scraper/)

Go ile yazılmış, yaklaşık 3.800 satırlık bağımsız bir komut satırı aracıdır. Program onu Docker ile ya da yerelde derlenmiş ikiliyle çağırır (`adimlar._bologna`).

Paket	Sorumluluk
`cmd/bologna/`	Giriş noktası.
`internal/cli/`	Komutlar (cobra): `migrate` (şema), `discover` (birim/program listesi), `scrape` (`-p/--program`, `--level`, `--all`, `--pool-details`, `--years`, `--fresh-within` ile kesilen taramayı sürdürme), `inspect` (bir sayfanın ayrıştırılmış hâli), `stats`. Program ilerlemeyi `ilerleme biten=… toplam=…` biçiminde basar; arayüz bunu tanır.
`internal/fetch/`	HTTP istemcisi: hız sınırı, tekrar deneme, 503/429'da bekleme, çerezler, ASP.NET form gönderimi (postback, ViewState), karakter kodlaması.
`internal/parse/`	HTML → Go yapıları, sayfa türü başına bir ayrıştırıcı: birimler, birim tanıtımı, program, ders planı, ders, tablolar, form, metin.
`internal/scraper/`	Tarama düzeni: seviye → birim → program → plan yılları → dersler; havuz derslerinin tekilleştirilmesi; devam.
`internal/store/`	PostgreSQL şeması (yabancı anahtarlı tablolar, görünümler), yazma, `scrape_runs`/`scrape_errors`; site başına şema (`search_path`) ve `site_info` ile şema sahipliği.

Veri modeli: `units` → `programs` → (`program_outcomes`, `program_courses`) → `courses` → (`course_learning_outcomes`, `course_weekly_topics`, `course_resources`, `course_assessments`, `course_workloads`, `course_outcome_matrix`, `course_program_outcomes`). Hazır görünümler: `v_curriculum`, `v_course_lo_po_matrix`, `v_course_outcome_matrix`. Ayrıntılar `bologna-scraper/README.md` ve `RAPOR.md`'de.

9. Çalıştırma motoru ve arayüz

9.1 Akış



Şekil 4. *Çalıştırma motoru: komuttan arayüze, kayda ve adım raporuna*

9.2 Alt süreç ve kayıt

- Python araçları `PYTHONUNBUFFERED=1` ve UTF-8 çıktı ile çalıştırılır; böylece çıktı tamponlanmadan, Türkçe karakterler bozulmadan gelir.
- API anahtarları yalnız ortam değişkeni olarak alt sürece verilir; kayda ve ekrana yazılmaz (komut gösteriminde gizlenir).
- Her çalışmanın tam kaydı `ciktilar/<profil>/merkez_kayitlari/<tarih>.log` dosyasına yazılır. Başarısız bir adımın son satırları ve kayıt yolu ayrıca gösterilir.
- Bir adımın birden çok komutu olabilir (ör. tüm site testinin parçaları, canlı doğrulamanın hazırla → çek → ölç → karşılaştır zinciri); her biri kayda başlığıyla yazılır.

9.3 İlerleme tanıma

Biçim	Örnek	Kim basar
`[i/n]`	`[946/2000] ✓ 270 başarılı …`	tester, doğrulama, metin, tarama, analiz, yerel model
girintili `i/n`	`  120/900 (3 kırık)`	link ve belge alt aşamaları
`N sayfa | kuyruk M`	`450 sayfa | kuyruk 1200`	keşif
`biten= toplam=`	`ilerleme biten=12 toplam=442`	Bologna

Araçlar paralel çalıştığı için sayfalar liste sırasıyla bitmez. Bu yüzden araçlar ilerleme olarak **tamamlanan iş sayısını** basar; motor da bir komut içinde görülen en büyük değeri tutar, böylece yüzde geri gitmez. Komutun ilk satırı ve önceden yapılmış işi atlayan sıçramalar "atlama" olarak işaretlenir ve hız hesabına girmez.

9.4 Kalan süre tahmini

`SureTahmini` (oran, zaman) gözlemlerini son 600 saniyelik kayan pencerede tutar ve kalan süreyi bu penceredeki hızdan hesaplar. Böylece başlangıçtaki hızlı atlamalar (önceden yapılmış sayfalar) ya da

geçici yavaşlamalar tahmini uzun süre bozmaz. Genel yüzde, adım sırası ve adım içi orandan hesaplanır.

9.5 Hata ve uyarı tanıma

`sorun_turu` satırları kalıplarla sınıflandırır: Python hata dökümü (`Traceback`), istisna son satırı, `[HATA]`, hata işaretleri, `level=ERROR`, `error:/fatal:/panic:` ve "no space left on device" **hata**; uyarı kalıpları **uyarı** sayılır. Hatalar ve uyarılar arayüzde ayrı sekmede listelenir; çift tıklayınca kayıtta o satıra gidilir.

9.6 Durdurma ve devam

Durdur'a basıldığında süreç önce nazikçe uyarılır (Windows'ta Ctrl+Break, diğerlerinde SIGINT). Araçlar bu sinyalde kısmi sonucu kaydeder. Süreç kapanmazsa tarayıcılar dahil bütün süreç ağacı zorla sonlandırılır. Aynı adım yeniden çalıştırıldığında araçlar kaldıkları yerden devam eder.

9.7 Arayüz performansı

Arayüz Tkinter ile yazıldı ve Windows'ta binlerce satırlık çıktı akarken de akıcı kalacak şekilde düzenlendi:

- Varsayılan **sade** görünüm yerleşik clam teması üzerine çizilir; görüntü tabanlı sv-ttk teması Windows'ta 5–7 kat yavaş ölçüldüğü için isteğe bağlıdır ("modern").
- Sayfalar bir kez kurulur, üst üste yığılır ve öne alınarak değiştirilir; arka plandaki sayfalar o anki boyutuna sabitlenir. Sayfa ve tema değişiminde Windows'ta çizim geçici olarak durdurulup tek seferde gösterilir (`_CizimiDondur`).
- Durum (hangi adım tamam, özet sayılar) arka plan iş parçacığında hesaplanır; büyük dosyalar (site haritası, `results.jsonl`) değişmedikçe yeniden okunmaz (dosya boyutu + zaman imzası önbelleği).
- Motor olayları kuyruktan 40 ms'lik bütçeyle işlenir; art arda gelen ilerleme olayları birleştirilir.

9.8 Adım raporları

Her adım bittiğinde `merkez/adim_raporu.py` `ciktilar/<profil>/adim_raporlari/<tarih>_<adım>.html` dosyasını yazar ve `index.html` dizinini günceller. Rapor; sonucu, süreyi, hata/uyarı sayısını, çalıştırılan komutları (anahtarlar gizli), hataları ve adımın ürettiği her dosyayı gösterir: HTML raporlara bağlantı, CSV'lerden ilk 500 satır aranabilir tablo, TXT'lerden 200 KB'a kadar metin, JSON'lardan öğe sayıları ve çalışma kaydının son 400 satırı. Böylece kendi raporu olmayan adımlar (kurulum, Bologna, ortam kontrolü) dahil her adımın sonucu görülebilir.

10. Veri ve dosya biçimleri

10.1 Çıktı klasörü

```
ciktilar/<profil>/
├─ kesif/                    site_haritasi.json, şablon kümeleri
├─ deneme_output/            deneme testi
├─ test_output/              results.jsonl, screenshots/, report.html/xlsx, summary.txt
├─ denetim/                  belge denetimi (documents_audit.csv, belge_raporu.html)
├─ linkdenetim/              kırık link (broken_links.csv, kirik_link_raporu.html)
├─ gorsel/, gorsel_ai/       görsel kusur, yerel model
├─ routes_verification.csv, clicks_verification.csv
├─ visual_audit/
│  ├─ denetim_cikti/         tarama.csv, ornekler.csv, ai_v2/, rapor/
│  ├─ temsilci_urls.json, metin_kontrol.csv
│  ├─ dogrulama_cikti/, etiket/, css_cikti/, oto_cikti/
├─ yeniden_cekim/, dogrulama/, dogrulama_css/   doğrulama çekimleri
├─ adim_raporlari/           her adımın her çalışması
├─ merkez_kayitlari/         tam çalışma kayıtları
└─ TESLIM/                   teslim paketi
```

10.2 Ana dosyalar

Programın ürettiği Excel CSV'leri BOM'lu UTF-8 ve ; ayraçlıdır; Türkçe Windows'ta Excel'de çift tıklayınca doğru açılır.

Dosya	Biçim	İçerik
`site_haritasi.json`	JSON	`url_list`: test edilecek adresler (program eski `pages` biçimini de okur).
`results.jsonl`	satır başına JSON	Sayfa başına: url, http_status, süre, security, turkish, accessibility, forms, mobil, konsol hataları, ekran görüntüsü yolları, görsel kusurlar, toplanan bağlantılar ve belgeler.
`broken_links.csv`	CSV	url, status, referrer (bağlantıyı veren sayfa), text (buton metni).
`documents_audit.csv`	CSV	url, status, ctype, size, source, flag.
`routes_verification.csv`	CSV	url, direct_status, parent_status, in_app, verdict.
`clicks_verification.csv`	CSV	kirik_url, referrer, referrer_status, buton_var, gidilen_url, verdict.
`tarama.csv`	CSV (;)	dosya, cihaz, sayfa, şablon, küme, ölçümler, bayraklar.
`ornekler.csv`	CSV (;)	yapay zekâya gidecek temsilci görüntüler: dosya, sayfa, cihaz, şablon.
`ai_v2/sonuclar.jsonl`	satır başına JSON	her (model, görüntü, dilim) cevabı: defects listesi, token kullanımı, hata, süre. Önbellek görevi görür.
`ai_v2/kapsam.json`	JSON	ilk model ve hakem için beklenen / tamam / yapılamadı / denenmedi / kota ve hata türleri.
`ai_v2/oylama.csv`	CSV (;)	karar (KESIN/INCELE), oy, bakan_model, tür, önem, dosya, dilim, modeller, örnek kanıt.
`dogrulama_cikti/sonuc.csv`	CSV (;)	karar, kaynak, tür, cihaz, url, sayfa, dilim, sayfa_kayiyor, ayrıntı, kanıt görüntüsü.

Dosya	Biçim	İçerik
`kok_neden.csv`	CSV (;)	sayfa_sayisi, cihaz, tür, seçici, en büyük taşma (px), örnek metin, örnek adres.
`oneri.css`	CSS	kurallar; her kuralın üstünde kaç sayfayı düzelttiği yorum olarak.

11. Ayarlar ve profiller

Ayarlar iki dosyada tutulur ve program içinde tek sözlükte birleşir:

- `profiller/<profil>.json`: siteye özgü her şey (adres, ad, keşif kuralları, test parametreleri, yerel ayar, User-Agent, yapay zekâ talimatındaki site tanımı ve dili, Bologna adresi). Başka bir bilgisayara taşınabilir; yollar repo köküne göreli tutulur.
- `ayarlar.json`: genel ayarlar (etkin profil, Python yolu, sağlayıcı anahtarları ve adresleri, modeller, veritabanı, görünüm). Git dışıdır.

Ayar	Varsayılan	Etkisi
Eşzamanlı sekme	4	Test, link, belge, doğrulama hızı ve sunucu yükü.
Sayfa zaman aşımı	30 sn	Yavaş sayfaların hata sayılma eşiği.
Parça boyutu	2.000	Tüm site testinin kaç sayfalık parçalara bölüneceği.
Tarayan modeller	`gemini:gemma-4-31b-it`	İlk model(ler); virgülle birden çok.
Hakem modeli	`gemini:gemini-3.5-flash-lite`	Şüpheli dilimlere ikinci görüş.
Dakikada istek	8	Ücretsiz kotalara takılmamak için hız sınırı.
Temsilci sayısı	3	Şablon/küme başına temsilci.
Görünüm	sade	"modern" sv-ttk temasını açar (yavaş).

12. Kalite güvencesi

12.1 Testler

`tests/test_merkez.py` 46 test içerir ve yalnız standart kütüphaneyle çalışır (araçların bağımlılıkları kurulmadan). Başlıca gruplar:

Grup	Neyi güvenceye alır
Ayarlar ve profiller	Kaydet/yükle, göreli yollar, yanlış türlerin yok sayılması, yeni profil.
Adımlar	Her adımın komutlarının üretilebilmesi; parça bölme; hakem turları; tıklama doğrulamanın kırık link CSV'sini kullanması.
Sözleşme testleri	Programın araçlara gönderdiği **her seçeneğin** aracın kaynak kodunda tanımlı olması; programdaki varsayılanların araçtakilerle aynı olması. Araçlar güncellendiğinde uyumsuzluğu yakalar.
İzleme	İlerleme biçimleri, sırasız ilerleme, kalan süre, hata/uyarı tanıma, ilk satırın atlama sayılması.
Raporlar	Link/belge raporu, Teyit sütunu, Excel CSV; adım raporu ve dizini.
Yapay zekâ	Talimatın değişmediği (eski sonuçlarla karşılaştırılabilirlik), ilerleme satırı, zaman aşımı, yeniden deneme turları, ilk model tamamlanmadan hakemin başlamaması.
Ana sayfa	Yol haritası ve kutucuklar; ilk model ve hakemin ayrı aşamalar olması.
Arayüz	Ekran varsa (Linux'ta Xvfb) pencerenin açılması ve sayfa geçişleri; ekran yoksa atlanır.

12.2 Sürekli entegrasyon

GitHub Actions (`.github/workflows/test.yml`) her değişiklikte iki iş çalıştırır: (1) Python testleri Ubuntu ve Windows'ta, Python 3.9 ve 3.12 ile (4 kombinasyon); (2) Bologna: `make lint`, PostgreSQL 16 servisiyle entegrasyon testleri ve derleme.

12.3 Dokümantasyon

Kullanım kılavuzu ve bu doküman `docs/kaynak/` altındaki betiklerden üretilir (`derle.sh`, `teknik_derle.sh`): Node.js docx ile Word, LibreOffice ile PDF; içindekiler sayfa numaraları iki geçişte bulunur. Kılavuzdaki adım listesi doğrudan programdan alınır (`veri_cikar.py`), böylece kılavuz programla çelişmez. Ekran görüntüleri ve bu dokümandaki diyagramlar da betikle yeniden üretilir.

13. Sınırlar ve geliştirme önerileri

13.1 Bilinen sınırlar

- **Keşif** yalnız bağlantı verilen sayfaları bulur; hiçbir yerden bağlantı verilmeyen sayfalar ve oturum açma gerektiren alanlar kapsam dışıdır.
- **Yapay zekâ** yalnız temsilcilere bakar; tek bir sayfanın içeriğinden doğan görsel hata, o sayfa temsilci değilse yapay zekâya gitmez. Kod kontrolleri ve link/belge denetimi bütün sayfaları kapsar.
- **Ücretsiz modeller** yavaştır ve hata verebilir (Gemma'da 500); bu yüzden yeniden deneme turları, hakem ve kapsam kontrolü vardır. Hakem onay oranı isabetin alt sınırıdır.
- **Canlı doğrulama** bulgunun bugünkü hâlini ölçer; site değiştiyse eski bulgu GÖRÜLMEDİ çıkabilir.
- **CSS önerisi** yalnız yana kaymayı ölçer; görsel olarak başka bir şeyi bozup bozmadığı karşılaştırma görüntülerinde gözle kontrol edilmelidir.
- **Dış bağlantılar** anlık olarak erişilemez olabilir; kırık link raporunda dış ve iç bağlantılar ayrı süzülebilir.

13.2 Geliştirme önerileri

- Oturum açma gerektiren sayfalar için profil başına giriş betiği.
- Bir önceki denetimle karşılaştırma raporu (yeni çıkan / düzelen bulgular).
- Erişilebilirlik denetiminin WCAG kurallarıyla genişletilmesi (renk kontrastı, klavye erişimi).
- Yapay zekâ analizinde ücretli toplu (batch) API'lerin Gemini için de kullanılması.
- Zamanlanmış (ör. aylık) otomatik denetim ve e-posta özeti.

Ek A. Adım kataloğu

Programdaki 39 adımın tamamı, gruplarına göre:

Grup	Adımlar
Hazırlık	Bağımlılıkları kur · Ortam kontrolü · Yapay zekâ sağlayıcılarını dene · Site keşfi · Keşif haritasını kullan
Web testi	Deneme testi · Tüm site kanıtlı testi · Hatalı sayfaları yeniden test et · Test raporunu yeniden üret
Denetimler	Belge denetimi · Kırık link denetimi · Şablon kümeleme · Görsel/UI kusur denetimi
Doğrulama	Kırık sayfa doğrulama (doğrudan erişim) · Buton tıklama doğrulama
Yapay zekâ denetimi	Yerel model incelemesi (Ollama) · 1. Kodla piksel taraması · 2a. Temsilci URL listesi · 2b. Kaydırarak yeniden çekim · 3a. Maliyet/süre tahmini · 3b. Yapay zekâ analizi (ilk model) · 3c. Hakem modeli · 4. İsabet ölçümü · 5. Metin kodlama denetimi · 6. Görsel denetim son raporu
Doğrula ve düzelt	7. Bulguları canlı sitede doğrula · 8a. Etiketleme sayfasını oluştur · 8b. Etiketleri işle · 9. CSS düzeltme önerisini dene · 10. Otomatik taşma düzeltme
Bologna	PostgreSQL'i başlat · Birim testleri · Programları listele · Tek program tara · Seviye tara · Tüm siteyi tara · Kesilen taramayı sürdür · Veritabanı istatistikleri
Teslim	Teslim paketini oluştur

Her adımın ayrıntılı açıklaması, tahmini süresi ve çıktı dosyaları Kullanım Kılavuzu'nun 6. bölümündedir; aynı bilgi `python web_denetim.py --liste` ile de alınabilir.

Ek B. Sözlük

Terim	Açıklama
SPA	Tek sayfa uygulaması: sayfalar tarayıcıda JavaScript ile çizilir (ör. React). Doğrudan adres erişimi sunucuda farklı davranabilir.
DOM	Tarayıcının sayfadan kurduğu öğe ağacı; ölçümler bunun üzerinde yapılır.
dHash	Görüntünün komşu piksel parlaklık farklarından üretilen kısa parmak izi; benzer görüntülerin parmak izleri arasındaki Hamming mesafesi küçüktür.
Mojibake	Karakter kodlaması hatasıyla bozulmuş metin (ör. ü yerine Ã¼).
Precision / recall	İsabet (bulunanların ne kadarı doğru) / yakalama (gerçek hataların ne kadarı bulundu).
Soft-404	Sunucu 200 dönmesine rağmen içeriği "bulunamadı" olan sayfa.
Hakem	İlk modelin şüpheli bulgularına ikinci görüş veren model.
JSONL	Her satırı ayrı bir JSON nesnesi olan dosya; yarıda kalan işlerde güvenle eklenebilir.
Batch API	İsteklerin toplu gönderilip sonuçların sonra alındığı, daha ucuz çağrı biçimi.